

INTRODUCTION A REACT

1. Rappels des fondamentaux des sites web statiques

Une page web statique est une page web dont le contenu ne varie pas en fonction des caractéristiques de la demande, c'est-à-dire qu'à un moment donné tous les internautes qui demandent la page reçoivent le même contenu.

À l'inverse, une page web dynamique est générée à la demande et son contenu varie en fonction des caractéristiques de la demande (heure, adresse IP de l'ordinateur du demandeur, formulaire rempli par le demandeur, etc.) qui ne sont connues qu'au moment de sa consultation.

2. Différence entre front end et back end

a. Le back-end :

Le back-end est une partie importante du développement, sans laquelle un site web ou une application ne pourrait marcher. Même s'il est indispensable, le back-end est un ensemble d'opérations qui ne sont pas visibles pour les visiteurs. Le back-end est hiérarchisé en trois parties, à savoir le serveur ou hébergeur, l'application et la base de données.

Le développeur back-end utilise des langages de programmation comme Python, PHP, Ruby pour mettre en place et configurer le serveur. Ces outils vont lui permettre de conserver, traiter et modifier des informations. Il doit s'assurer que ces dernières soient toujours à jour. Pour rendre ces langages de programmation encore plus pratiques, les développeurs vont les améliorer par des frameworks comme Symfony, Ruby on Rails, CakePHP ou encore Codelgniter. Ces outils vont rendre le développement plus rapide et plus sécurisé.

Le développeur back-end a ainsi pour mission de se charger de la création et de la gestion de tous les éléments invisibles pour l'utilisateur final. C'est donc lui qui est responsable de toutes les fonctionnalités du site ou de l'application. Il est également responsable de la création de la base de données qui va permettre, entre autres, de retenir les informations fournies par les utilisateurs. Par exemple, le développeur back-end va utiliser les bases de données pour retrouver les identifiants et les mots de passe utilisés par les clients pour se connecter. Il est possible de se former à ce métier en passant par une formation en web development voire même par une formation à Python.

b. Le front-end :

Le terme front-end désigne les éléments d'un site ou d'une application que les utilisateurs voient à l'écran et avec lesquels ils vont interagir. À titre d'exemple, tout ce que les internautes vont voir sur un site internet, c'est une combinaison de HTML, CSS et JavaScript. Ce sont ces langages de programmation front-end qui seront interprétés par le navigateur.

En général, le front-end se compose d'un design et d'un code HTML, CSS JavaScript et jQuery. Le design est créé par le web designer qui va réaliser des maquettes graphiques avec des outils dédiés, comme Photoshop ou Fireworks.

c. Langages de programmation :

Front end	HTML, CSS, Javascript
Back end	Python, Java, PHP, C++

3. Présentation des Frameworks front end

Logo	Lien GitHub	Compagnies qui utilisent le Framework
	React Framework	• Dropbox • Instagram • Airbnb • Discord • Walmart • Pinterest
	Angular Framework	• Microsoft Office • Deutsche Bank • Mixer • Santander • Gmail • Forbes
	Vue.js Framework	• Nintendo • Passionate People • FindlayWebTech • Adobe • BridgeU • Gitlab
	Ember.js	• Twitch • LinkedIn • Accenture • Square • DigitalOcean • Groupon
	jQuery	• JPMorgan Chase • QTC Management • Wells Fargo • Edward Jones • The Walt Disney
	Semantic-UI Framework	• Snapchat • Accenture • Kmong • Modanisa • Digital Services • Aumni
	Backbone.js	• Racepoint Global • McCorkell & Associates Pty. Limited • AMC Networks Inc • University of California
	Preact Framework	NA

	Svelte Framework	• Capital One • Snap! Raise • GoGuardian
	Foundation	• Harvard University

4. Notion d'application web monopage SPA (Single Page Application)

Une application monopage ou SPA (« Single-page application » en anglais) est une implémentation d'application web qui ne charge qu'un seul document web, puis met à jour le contenu du corps de ce document via des API JavaScript telles que XMLHttpRequest et Fetch lorsqu'un contenu différent doit être affiché.

Cela permet donc aux utilisateurs d'utiliser des sites web sans charger de nouvelles pages entières à partir du serveur, ce qui peut entraîner des gains de performances et une expérience plus dynamique, avec certains inconvénients tels que le référencement (SEO), plus d'efforts nécessaires pour maintenir l'état, mettre en œuvre la navigation et mesurer les performances.

5. Single Page Application (SPA) vs Multipage Application (MPA)

i. La rapidité :

La vitesse est un facteur important ici – la capacité d'attention des gens est de plus en plus courte et nous devenons de plus en plus impatients. Le SPA se charge plus rapidement. Pourquoi ? Comme il charge la majorité des ressources de l'application une seule fois. La page ne se recharge pas entièrement chaque fois que l'utilisateur demande une nouvelle donnée.

Le MPA est plus lent car le navigateur doit recharger la page entière à partir de zéro chaque fois que l'utilisateur souhaite accéder à de nouvelles données ou se déplace vers une autre partie du site Web. Le temps de chargement optimal d'un site Web est de 0,4 seconde. Si votre site Web ou votre application contient beaucoup d'images, le choix d'un SPA est une option plus sûre.

ii. Couplage :

SPA est fortement découplé, ce qui signifie que le frontend et le backend sont séparés. Les applications d'une seule page utilisent des API développées par des développeurs côté serveur pour lire et afficher des données. Dans les AMP, le frontend et le backend sont plus interdépendants. Tout le codage est généralement hébergé dans un seul projet.

iii. Optimisation du moteur de recherche :

L'une des faiblesses du SPA est le référencement. Malheureusement, ils ne sont pas aussi conviviaux pour le référencement que les MPA. C'est principalement parce que la plupart des applications d'une seule page sont exécutées sur JavaScript, que la plupart des moteurs de recherche ne prennent pas en charge. Les pages Web sont indexées par "crawling" ou "spidering". Les robots des moteurs de recherche téléchargent les fichiers HTML de la page, ce qui facilite le classement des pages Web HTML statiques.

Le MPA permet un meilleur positionnement du site Web, car chaque page peut être optimisée pour un mot-clé différent. De plus, des balises méta peuvent être incluses sur chaque page, ce qui a un impact positif sur le classement de Google.

iv. Expérience utilisateur :

Une bonne expérience utilisateur n'est plus une option mais une exigence. Les SPA sont plus adaptés aux mobiles, ce qu'il convient de rappeler car une grande partie du trafic provient d'appareils mobiles. Même Google a commencé à donner la priorité à l'expérience mobile sur le bureau. Les cadres appliqués dans le développement SPA vous permettent de développer des applications mobiles.

Les MPA, d'autre part, permettent une meilleure architecture de l'information. Vous pouvez créer autant de pages que nécessaire et vous pouvez inclure autant d'informations sur une page que nécessaire sans aucune limite. La navigation est claire, de sorte que l'utilisateur peut facilement se repérer sur le site Web, ce qui a un impact positif sur son expérience.

v. Sécurité :

Cela ne vous surprendra probablement pas, mais plus le site Web est grand, plus il faut d'efforts pour le sécuriser - pensez aux applications multipages. Si vous optez pour une AMP, vous devrez alors sécuriser chaque page Web. Dans un SPA, tout ce que vous avez à faire pour assurer la sécurité de votre page est de sécuriser les points de terminaison de données plus rapidement, mais pas nécessairement plus sûrs. Les SPA sont plus sujets aux attaques de pirates, car ils fonctionnent sur JavaScript, qui n'effectue pas de compilation de code, ce qui le rend plus vulnérable aux logiciels malveillants.

vi. Processus de développement :

L'un des plus grands avantages des SPA est le code backend réutilisable. Si vous pensez que le code réutilisable équivaut à moins de travail, alors vous avez raison. Vous pouvez appliquer le même code que vous avez utilisé dans votre application Web à votre application mobile native. C'est une information importante, car les applications et les sites Web sont fréquemment utilisés sur les appareils mobiles - ce qui n'est pas surprenant puisque la plupart d'entre nous sommes constamment en fuite.

Grâce à la division claire entre le frontend et le backend, les deux parties peuvent être développées simultanément, ce qui accélère l'ensemble du processus de développement. Les MPA prennent plus de temps à se développer car dans la plupart des cas, le côté serveur doit être codé dès le début.

vii. Dépendance JavaScript :

SPA vit et respire JavaScript, ce qui peut être problématique. De plus en plus de moteurs de recherche ont commencé à prendre en charge JavaScript, mais avec des résultats variables. Le niveau de support dépend fortement du Framework JS utilisé. Si l'application est exécutée sur un navigateur avec JavaScript désactivé, cela peut entraîner des problèmes de fonctionnalité de l'application, ce qui peut entraîner des taux de rebond plus élevés et une conversion plus faible. La dépendance à JavaScript contribue également à ses problèmes d'optimisation SEO et de sécurité. Les AMP peuvent être construites sans aucune dépendance JavaScript.

viii. SPA VS MPA – réflexions finales :

Aucune des architectures décrites ci-dessus n'est parfaite - elles ont toutes deux des avantages et des inconvénients. SPA gagne en termes de rapidité et de réutilisabilité du code, ce qui peut être appliqué pour développer votre application mobile, mais il présente des lacunes en matière d'optimisation SEO. L'utilisation d'un MPA vous aidera à vous classer plus haut dans Google et est plus évolutif mais beaucoup plus lent que les SPA.

Lorsque vous décidez quelle option choisir, vous devez toujours garder à l'esprit vos objectifs commerciaux ainsi que vos exigences. Les SPA sont mieux utilisés dans les applications de réseaux sociaux, les plates-formes SaaS - partout où les classements SEO ne sont pas un facteur décisif.

Les MPA sont mieux utilisés dans les applications de commerce électronique, les catalogues d'entreprise et les places de marché. Si vous êtes une grande entreprise qui propose une grande variété de produits, une MPA est le meilleur choix pour vous.

6. Fonctionnement d'une application web de type SPA

Le client se connecte d'abord au serveur et obtient le contenu de la page, qui correspond principalement au code HTML, au CSS et à un bundle JavaScript, contenant tous les JavaScripts nécessaires pour exécuter la logique de l'application.

L'action d'un utilisateur déclenche l'exécution de JavaScript associés, qui à leur tour demandent des données au serveur par le biais d'appels Ajax. Les données sont généralement fournies dans un format JSON et ne déclenchent pas de rafraîchissement complet de la page Web.

Dans une application multi-tiers (MPA) impliquant des serveurs back-end, le serveur web peut se contenter de télécharger le paquet initial de HTML/CSS/JavaScript. Tous les appels Ajax ultérieurs se font directement entre le navigateur et les services back-end.

7. Avantages et inconvénients d'une application web de type SPA

i. Avantages d'une application à page unique (SPA) :

Les applications à page unique sont connues pour tirer parti des mises en page répétitives et du contenu à la demande. Comme elles tirent parti de ces deux concepts, elles sont beaucoup plus efficaces, coûtent nettement moins cher que les applications traditionnelles et consomment également moins d'énergie. Voici quelques avantages à l'utilisation des applications à page unique.

- **Vitesse accrue**

La vitesse doit être l'un des plus grands avantages d'opter pour le développement d'une application à page unique. Elles sont beaucoup plus rapides que les applications web traditionnelles car elles peuvent charger de nouvelles informations dans une seule page à la demande du client au lieu de relier plusieurs pages HTML à l'architecture du site.

Comme l'application n'a pas besoin d'envoyer constamment des requêtes au serveur et d'attendre de télécharger une nouvelle page à partir de zéro, une application à page unique est beaucoup plus facile.

- **Expérience utilisateur (UX)**

Comme nous l'avons vu précédemment, les applications à page unique sont les leaders en matière d'expérience utilisateur pratique et directe. La plupart des utilisateurs sont souvent déroutés et irrités de devoir cliquer sur de nombreux liens dans une application traditionnelle pour arriver là où ils veulent.

Les applications à page unique éliminent ce problème en fournissant tout le contenu nécessaire dans une seule page déroulante. Cela fait également de la SPA un choix idéal pour les entreprises dont un grand nombre d'utilisateurs accèdent à leurs applications via des navigateurs mobiles.

- **Capacités de mise en cache**

Une application à page unique est capable de mettre en cache les données locales plus efficacement. Comme l'application à page unique n'envoie qu'une seule requête au serveur et stocke toutes les données qu'elle reçoit, elle peut également utiliser ces données pour travailler hors ligne.

L'avantage pour l'utilisateur est qu'il peut facilement continuer à travailler sur son projet même si sa connexion internet est faible ou inexistante. Les données locales peuvent être synchronisées avec le serveur dès que la connexion est rétablie.

- **Débogage avec Chrome**

Le débogage des SPA avec Chrome est beaucoup plus facile que le débogage de n'importe quelle application à pages multiples. De plus, comme les SPA sont construites avec des frameworks tels que Angular et React, ces frameworks ont leurs propres outils de débogage Chrome, ce qui facilite grandement leur débogage.

ii. Inconvénients d'une application à page unique (SPA) :

- **Historique du navigateur**

L'un des inconvénients d'une expérience utilisateur transparente et de l'absence de chargement de page avec les applications à page unique est que les SPA n'enregistrent pas les sauts des utilisateurs entre les différents états.

Cela signifie que lorsque l'utilisateur appuie sur le bouton « retour », il ne peut pas revenir en arrière. Le bouton retour ne ramène l'utilisateur qu'à la page précédente et non à l'état précédent dans l'application.

Il existe une solution à ce problème en utilisant l'API HTML 5 History, intégrée aux frameworks SPA. Cette API permet aux développeurs d'accéder à l'historique de navigation du navigateur via JavaScript.

- **Optimisation du référencement**

L'un des plus grands signaux d'alarme qui empêchent le développeur d'explorer les applications à page unique est la difficulté d'optimisation du référencement.

La principale raison est que les applications à page unique semblent fonctionner en JavaScript, et en fonction de la demande provenant du côté client, elles téléchargent des données.

Les différentes pages ne sont pas accompagnées d'adresses URL uniques et l'URL ne change pas beaucoup. Comme la plupart des pages ne peuvent être trouvées et analysées par les robots de recherche, il n'est pas facile d'optimiser les sites Web pour les moteurs de recherche.

Les développeurs doivent s'assurer que Google indexe tous les fichiers JavaScript pour que cela soit possible. Les développeurs doivent également confirmer que dans le schéma URL, le site Web utilise le mode HTML5.

- **Problèmes de sécurité**

Par rapport aux applications multi pages, les applications monopages sont plus vulnérables aux attaques par cross-site Scripting. En effet, les pirates peuvent injecter des scripts côté client dans les applications web en utilisant le XSS.

L'exposition d'informations sensibles est l'un des problèmes de sécurité qui se posent ici. Si les développeurs n'ont aucune idée du type de données présentes dans le chargement initial de la page, il y a de fortes chances qu'ils envoient ces données aux utilisateurs, alors qu'elles auraient dû rester cachées.

Dans le scénario habituel, le navigateur n'affiche pas la totalité de l'application à page unique, ce qui constitue un faux signal de sécurité. Le fait de ne pas avoir accès au niveau fonctionnel est une raison supplémentaire pour laquelle les applications monopages manquent de sécurité.

Étant donné que les développeurs retirent la logique et les fonctionnalités du serveur pour les déplacer vers le client, il est assez facile d'offrir l'accès aux fonctions aux clients, ce qui ne devrait pas être le cas de la bonne façon.

Les applications à page unique conviennent parfaitement au développement de plateformes robustes avec des volumes de données mineurs.

Une application à page unique est parfaite comme base pour le développement futur d'applications mobiles. Les SPA visent à offrir une expérience utilisateur exceptionnelle en essayant de reproduire un environnement naturel dans le navigateur : pas de rechargement de page ni de temps d'attente supplémentaire.

Ce modèle est remarquable pour les réseaux sociaux, les plateformes SaaS et les communautés fermées où l'optimisation des moteurs de recherche n'a pas d'importance.

8. Présentation de React :

React (Ou ReactJS, React.js) est une bibliothèque de source ouverte Javascript pour construire des composants de l'interface réutilisable. Il était créé par Jordan Walke, un ingénieur en informatique à Facebook. La personne est affectée par XHP (une plate-forme HTML pour PHP). Le React a d'abord été déployée pour l'application Newsfeed de Facebook en 2011, puis pour Instagram.com en 2012. Il était à code source ouvert (open-sourced) sur JSConf US en mai 2013.

9. Intérêt du développement des composants avec React (Components) :

La caractéristique principale de React est la composition de composants. Les composants écrits par des personnes différentes devraient fonctionner correctement ensemble. Il faut tenir à ce que vous puissiez ajouter des fonctionnalités à un composant sans que ça impacte le reste de la base de code.

Les composants sont souvent décrits comme de « simples fonctions » mais ils doivent être bien plus que ça pour être utiles. Avec React, les composants décrivent un comportement composable, ce qui inclut le rendu, le cycle de vie et l'état local. Certaines bibliothèques externes telles que Relay ajoutent d'autres responsabilités aux composants comme la description des dépendances de données. Il est possible que ces idées reviennent dans React sous une forme ou une autre.

10. Architecture d'une application React :



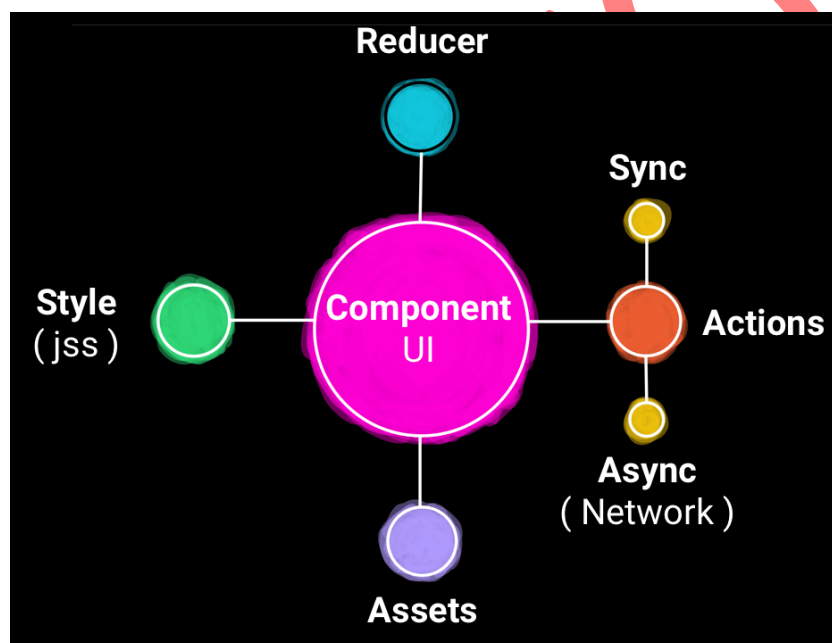
ESLint analyse statiquement votre code pour trouver rapidement les problèmes. Il est intégré à la plupart des éditeurs de texte et vous pouvez exécuter ESLint dans le cadre de votre pipeline d'intégration continue.

Approche CI/CD : L'approche CI/CD permet d'augmenter la fréquence de distribution des applications grâce à l'introduction de l'automatisation au niveau des étapes de développement des applications. Les principaux concepts liés à l'approche CI/CD sont l'intégration continue, la distribution continue et le déploiement continu. L'approche CI/CD représente une solution aux problèmes posés par l'intégration de nouveaux segments de code pour les équipes de développement et d'exploitation (ce qu'on appelle en anglais « integration hell », ou l'enfer de l'intégration).

Middleware (ou intergiciel en français) est un logiciel qui agit comme une passerelle entre les autres applications, outils et bases de données pour offrir aux utilisateurs des services unifiés. Il est souvent décrit comme le ciment qui lie ensemble les différents appareils et plateformes logicielles.

React-tracking : Bibliothèque de suivi spécifique à React, utilisable en tant que composant d'ordre supérieur (en tant que `@decorator` directement), ou en tant que React Hook (fonction qui vous permettent de « vous connecter » à l'état React et aux fonctionnalités de cycle de vie des composants fonctionnels).

Misc : Divers composants React utilisés.



11. Utilisation de CRA (Create React App) :

Create React App va vous permettre de générer un squelette de code pour votre application. Il embarque un certain nombre d'outils préconfigurés, tels que Webpack, Babel et ESLint, afin de vous garantir la meilleure expérience de développement possible.

12. Notion de CRA :

Les développeuses et développeurs utilisent des outils automatisés pour créer une base de code, qui dispose des outils essentiels déjà préconfigurés. Pour vous citer quelques-unes des fonctionnalités de ces outils, ils permettent de :

- Gérer les différentes dépendances (bibliothèques) utilisées par notre app ;
- Optimiser le chargement de notre code dans les navigateurs ;
- Importer du CSS et des images ;
- Gérer les différentes versions de JavaScript ;
- Faciliter l'expérience de développement, en rechargeant la page lorsque le code est modifié.

13. Installation de CRA :

Pour manipuler Create React App ici, nous allons avoir besoin d'un gestionnaire de paquet (package manager) directement dans le terminal. Par exemple **yarn** (Gestionnaire de packages qui se double d'un gestionnaire de projet) ou **npm** (Node Package Manager).

Installer le Node.js qui contient déjà le npm. Ensuite installer create-react-app en tapant dans votre invite de commande :

```
npm install -g create-react-app
```

14. Lancement de CRA :

```
create-react-app mon-dossier-de-travail  
cd mon-dossier-de-travail  
npm start
```

15. Prise en main de CRA :

Pour commencer, placez-vous dans le dossier où vous voulez créer votre projet.
Pour initialiser votre projet, nous allons faire :

```
create-react-app la-maison-jungle
```

ou

```
npx create-react-app la-maison-jungle
```

npx est la commande conseillée par Create React App pour initialiser un projet, sans avoir à installer CRA globalement, ou localement.

Maintenant que votre projet est initialisé, il est temps de vous plonger dedans et de partir à l'exploration des fichiers créés. On rentre dans le projet avec la commande (dans votre terminal) :

```
cd la-maison-jungle
```

Ensuite :

```
npm start
```

Ensuite :

Commande	Signification
Node -v ou --version	Version de Node
Npm -v ou --version	Version de npm

Create-react-app --version	Version de create-react-app
Npm ls	Afficher la liste des packages créés

À partir de là, vous pouvez ouvrir votre éditeur de texte préféré (comme VS Code par exemple) pour jeter un œil. Voici un exemple :

Votre environnement de travail est particulièrement important. Vous devez vous sentir à l'aise avec les outils que vous utilisez. VS Code vous permet d'installer des extensions qui vous offrent des fonctionnalités supplémentaires. Typiquement, pour un projet React, les extensions Prettier et ESLint sont très utiles !

Vous trouverez trois dossiers :

- **node_modules** : c'est là que sont installées toutes les dépendances de notre code. Ce dossier peut vite devenir très volumineux.
- **public** : dans ce dossier, vous trouverez votre fichier index.html et d'autres fichiers relatifs au référencement web de votre page.
- **src** : vous venez de rentrer dans le cœur de l'action. L'essentiel des fichiers que vous créerez et modifierez seront là.

Et faisons maintenant un petit tour des fichiers importants :

- **package.json** : situé à la racine de votre projet, il vous permet de **gérer vos dépendances** (tous les outils permettant de construire votre projet), vos scripts qui peuvent être exécutés avec yarn, etc. Si vous examinez son contenu, vous pouvez voir des dépendances que vous connaissez : React et ReactDOM :
 - vous y trouverez react-scripts, créé par Facebook, qui permet d'installer Webpack, Babel, ESLint et d'autres pour vous faciliter la vie ;
- dans /public, vous trouvez index.html. Il s'agit du template de votre application. Il y a plein de lignes de code, mais vous remarquez <div id="root"></div> ? Comme dans les chapitres précédents, nous allons y ancrer notre app React ;
- dans /src, il y a index.js qui permet d'initialiser notre app React ;
- et enfin, dans /src, vous trouvez App.js qui est notre premier composant React.

Je vois **dependencies** et **devDependencies**. Mais c'est quoi la différence entre les deux ?

En fait, les **dependencies** sont des dépendances qui sont nécessaires à notre code pour fonctionner, aussi bien en local qu'en production. En revanche, les **devDependencies** sont uniquement nécessaires pour la phase de développement. Les deux sont installées depuis votre package manager, pour nous yarn, en précisant une option différente : yarn add pour les dependencies, et yarn add --dev pour les devDependencies.

Deux fichiers que nous n'utiliserons pas directement mais qui ne font pas de mal à garder :

- le **README.md** qui permet d'afficher une page d'explication si vous mettez votre code sur GitHub, par exemple ;
- et le fichier **.gitignore** qui précise ce qui ne doit pas être mis sur GitHub, typiquement le volumineux dossier des node_modules.

La base de code qui a été initialisée avec Create React App contient du code et des outils qui ne seront pas exploités dans ce cours, notamment tout ce qui concerne la partie de tests. Les fichiers que je n'ai pas mentionnés au-dessus, vous pouvez les supprimer. Ça vous permettra de garder ce qui est essentiel.